In [4]:
```
#Array implementation of stack
class Stack:
    def __init__(self,capacity):
        self.st=[]
        self.capacity=capacity

    def push(self,x):
        if self.isFull():
            return f"Stack Overflow"
        else:
            self.st.append(x)
            return f"{x} is inserted"

    def pop(self):
        if self.isEmpty():
            return f'Stack Underflow'
        else:
            return f'{self.st.pop()} is removed'


    def top(self):
        return f'{self.st[-1]} is the peek element'

    def isFull(self):
        return len(self.st)==self.capacity

    def isEmpty(self):
        return len(self.st)==0

    def size(self):
        return f'Stack size is :{len(self.st)}'

    def display(self):
        if self.isEmpty():
            return f"Stack underflow"
        return self.st

obj = Stack(5)

print(obj.push(10))
print(obj.push(20))
print(obj.push(30))
print(obj.push(40))
print(obj.push(50))
print()
print(obj.push(60))      # Stack Overflow
print()

print(obj.display())
print()

print(obj.pop())
print(obj.pop())
print()

print(obj.display())
print()
```

```
print(obj.top())
print()

print(obj.size())
print()

print(obj.isEmpty())
print()

print(obj.isFull())
```

```
10 is inserted
20 is inserted
30 is inserted
40 is inserted
50 is inserted

Stack Overflow

[10, 20, 30, 40, 50]

50 is removed
40 is removed

[10, 20, 30]

30 is the peek element

Stack size is :3

False

False
```

In [ ]:
```
#Array implementation of queue
class Queue:
    def __init__(self,capacity):
        self.q=[]
        self.capacity=capacity

    def enqueue(self,x):
        if self.isFull():
            return f'Queue overflow'
        else:
            self.q.append(x)
            return f'{x} inserted successfully'

    def dequeue(self):
        if self.isEmpty():
            return 'Queue underflow'
        else:
            return f'{self.q.pop(0)} is removed'

    def front(self):
        if self.isEmpty():
            return "Queue underflow"
        return f'{self.q[0]} is the front element'

    def rear(self):
        if self.isFull():
```

```
            return "Queue overflow"

        return f"{self.q[-1]} is the rear element"

    def isFull(self):
        return len(self.q)==self.capacity

    def isEmpty(self):
        return len(self.q)==0

    def size(self):
        return f"Queue size is:{len(self.q)}"

    def display(self):
        if self.isEmpty():
            return 'Queue underflow'
        return self.q

obj = Queue(5)

print(obj.enqueue(10))
print(obj.enqueue(20))
print(obj.enqueue(30))
print(obj.enqueue(40))
print(obj.enqueue(50))
print()
print(obj.enqueue(60))      # Stack Overflow
print()

print(obj.display())
print()

print(obj.dequeue())
print(obj.dequeue())
print()

print(obj.display())
print()

print(obj.front())
print()
print(obj.rear())
print()

print(obj.size())
print()

print(obj.isEmpty())
print()

print(obj.isFull())
```

```
10 inserted successfully
20 inserted successfully
30 inserted successfully
40 inserted successfully
50 inserted successfully

Queue overflow

[10, 20, 30, 40, 50]

10 is removed
20 is removed

[30, 40, 50]

30 is the front element

50 is the rear element

Queue size is:3

False

False
```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click here for more info.

View Jupyter log for further details.

In [9]:
```python
#Implementing stack using queue
from collections import deque
class Stack:
    def __init__(self):
        self.q=deque()

    def push(self,x):
        self.q.append(x)

        for i in range(len(self.q)-1):
            self.q.append(self.q.popleft())

    def pop(self):
        if len(self.q)==0:
            return -1

        return self.q.popleft()

    def top(self):
        if len(self.q)==0:
            return -1
        return self.q[0]

    def empty(self):
        return len(self.q)==0

obj = Stack()
```

```
obj.push(10)
obj.push(20)
obj.push(30)

print(obj.pop())      # 30
print(obj.top())      # 20
print(obj.empty())    # False
```

```
30
20
False
```

In [10]:
```
#Implement queue using stack
class Queue:
    def __init__(self):
        self.s1=[]
        self.s2=[]

    def enqueue(self,x):
        self.s1.append(x)

    def dequeue(self):
        if len(self.s2)==0:
            while len(self.s1)!=0:
                self.s2.append(self.s1.pop())
        if len(self.s2)==0:
            return -1
        return self.s2.pop()

    def front(self):
        if len(self.s2)==0:
            while len(self.s1)!=0:
                self.s2.append(self.s1.pop())
        if len(self.s2)==0:
            return -1

        return self.s2[-1]

    def empty(self):
        return len(self.s1)==0 and len(self.s2)==0

obj = Queue()

obj.enqueue(10)
obj.enqueue(20)
obj.enqueue(30)

print(obj.dequeue())   # 10
print(obj.front())     # 20
print(obj.empty())     # False
```

```
10
20
False
```

In [8]:
```
#Valid paranthesis
class solution:
    def ValidParanthesis(self,s):
        n=len(s)
```

```
        if n%2!=0:
            return False
        st=[]

        for ch in s:
            if ch=='(' or ch=='{' or ch=='[':
                st.append(ch)
            else:
                if len(st)==0:
                    return False
                top=st.pop()
            if ch==')' and top!='(':
                return False
            if ch=='}' and top!='{':
                return False
            if ch==']' and top!='[':
                return False
        return len(st)==0
obj=solution()
res=obj.ValidParanthesis("{[()]}")
print(res)
```

```
True
```